

An Experimental Analysis of the Fast Fourier Transform

Trevor Ritchie

Applied Algorithms

2026-04-28

1. Problem / Research Overview

The fundamental problem addressed in this research is the computational efficiency of the Discrete Fourier Transform (DFT). The DFT is essential for transforming a signal from the time domain to the frequency domain, enabling spectral analysis. However, a "brute-force" implementation of the DFT has an asymptotic time complexity of $O(n^2)$, where n is the number of samples. This quadratic growth becomes computationally prohibitive for large datasets common in real-time signal processing and high-fidelity audio.

This project implements and experimentally analyzes the Fast Fourier Transform (FFT), specifically the Cooley-Tukey algorithm. The FFT reduces the complexity to $O(n \log n)$ by leveraging a "Divide-and-Conquer" approach, recursively decomposing a DFT of size n into two DFTs of size $n/2$. The objective is to empirically demonstrate this performance gap and apply the optimized algorithm to physiological and musical signals.

2. Implementation and Technical Challenges

To fully explore the computational differences, I implemented three distinct versions of the transform. The baseline is a brute-force DFT, constructed with nested loops derived directly from the summation formula. To improve upon this, I developed a recursive Cooley-Tukey FFT that leverages a radix-2 approach, splitting the signal into even and odd indices and utilizing the symmetry and periodicity of twiddle factors to combine results in linear time. Finally, to eliminate the overhead associated with recursion, I created an iterative Cooley-Tukey FFT. This version employs bit-reversal permutation to pre-arrange the input data, allowing the core "butterfly" operations to proceed sequentially.

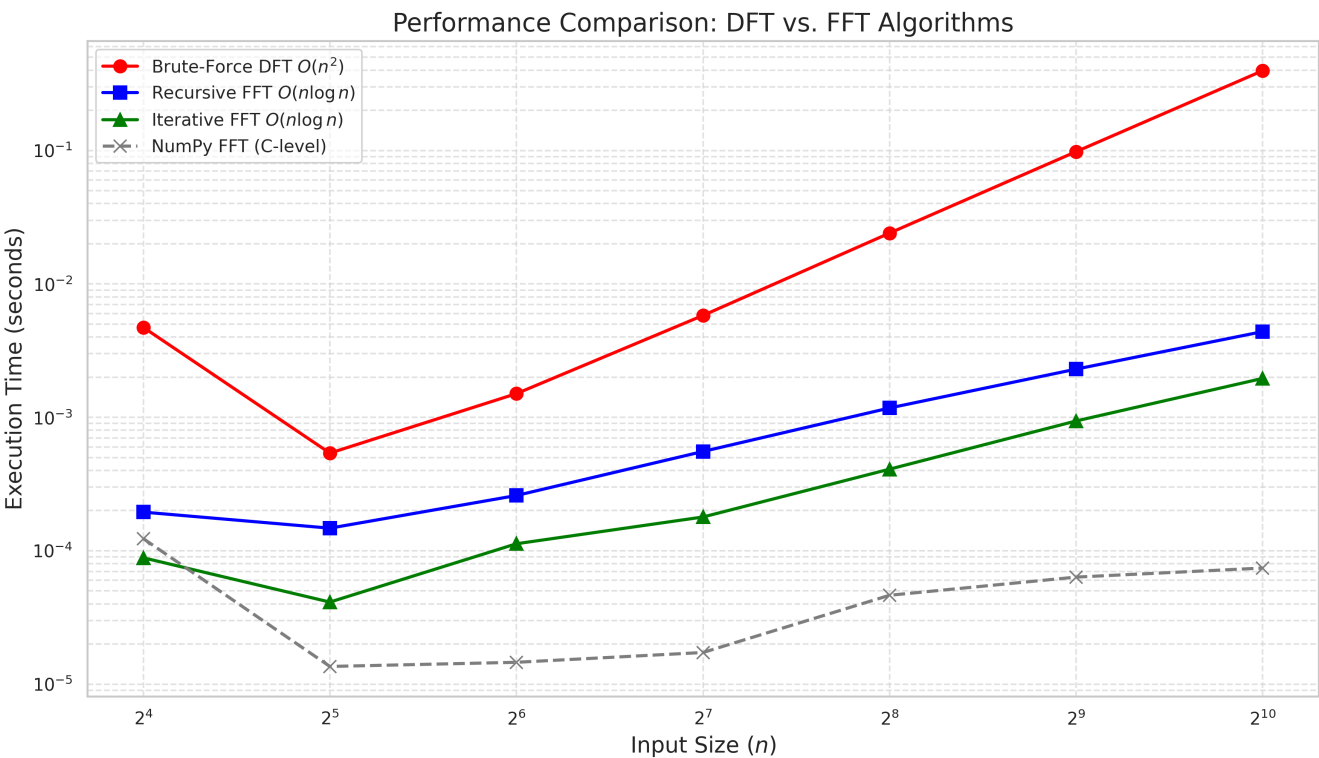
A primary technical hurdle was accommodating input signals whose lengths were not exact powers of two, a strict requirement for the radix-2 algorithm. I resolved this by introducing a zero-padding utility that gracefully extends any input array to the nearest power of two before processing. Additionally, accurately benchmarking these algorithms (particularly against highly

optimized libraries like NumPy) required precision. I utilized `time.perf_counter()` to guarantee high-resolution timing measurements.

I have addressed the feedback from the Project Update, which emphasized the necessity of rigorous data analysis and execution records. I developed a benchmarking script `src/benchmark.py`. This suite systematically records the execution times for the DFT, both FFT implementations, and the NumPy baseline across a spectrum of input sizes ranging from $n = 2^4$ to $n = 2^{10}$. These findings are visualized on log-log scales to unambiguously distinguish their respective complexity classes.

3. Performance Evaluation: DFT vs. FFT

The empirical performance data, illustrated in the `fft_vs_dft_performance.png` plot, clearly demonstrates the predicted theoretical behaviors. The brute-force DFT exhibits steep quadratic growth, becoming noticeably sluggish by $n = 1024$. In contrast, both the recursive and iterative FFT implementations closely trace the much flatter $O(n \log n)$ curve, maintaining high efficiency as the input size expands. Notably, the iterative FFT demonstrated a marginal performance edge over its recursive counterpart due to reduced call-stack overhead.

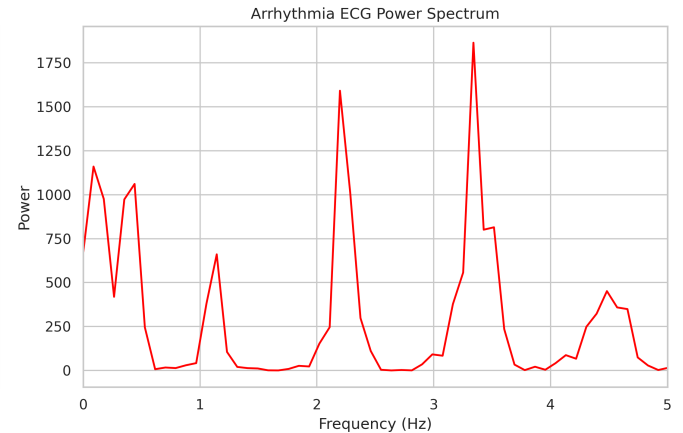
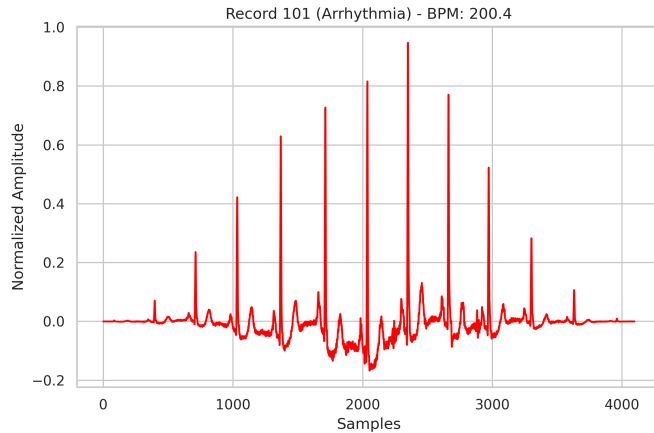
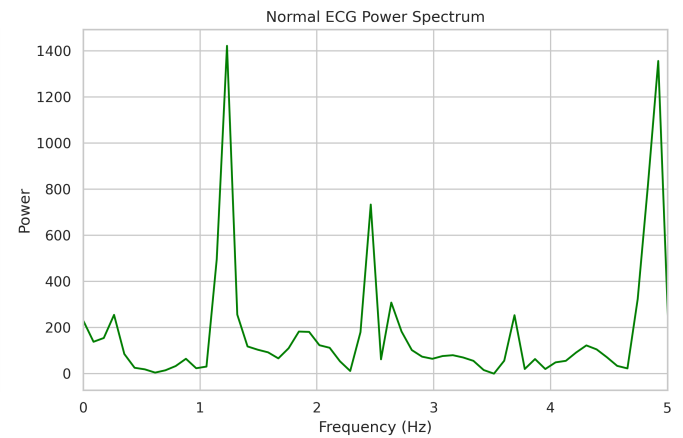
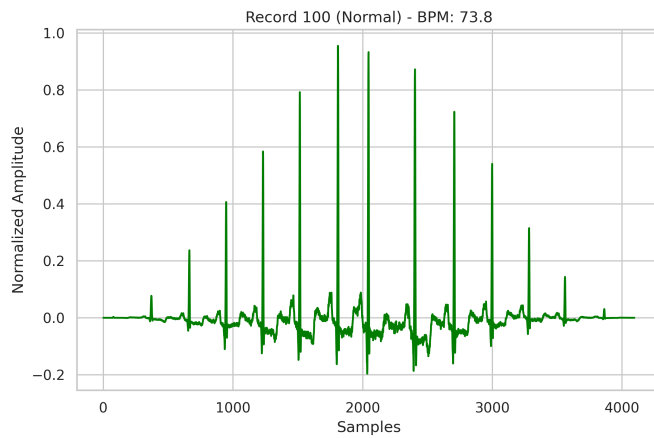


Performance Benchmarking Data

Input Size (n)	DFT Time (s)	FFT (Rec) (s)	FFT (Iter) (s)	NumPy FFT (s)	Speedup (DFT/Rec)
16	0.004723	0.000195	0.000089	0.000123	24.24x
32	0.000540	0.000148	0.000041	0.000014	3.65x
64	0.001506	0.000260	0.000113	0.000015	5.80x
128	0.005804	0.000555	0.000179	0.000017	10.46x
256	0.024039	0.001176	0.000409	0.000046	20.45x
512	0.098024	0.002298	0.000940	0.000063	42.65x
1024	0.396292	0.004390	0.001958	0.000074	90.27x

4. Real-World Application: ECG Analysis

To validate the practical utility of these implementations, I evaluated them against real-world datasets, moving beyond abstract benchmarking to extract meaningful information from the signals. For real-world testing, I analyzed ECG recordings (specifically Record 100, a normal heartbeat, and Record 101, an arrhythmia) sourced from the MIT-BIH Arrhythmia Database. To prepare the signals, I applied detrending to remove the mean, normalized the amplitude, and applied a Hann window to reduce spectral leakage over 4096-sample segments. By analyzing the power spectrum within the 0.5-5.0 Hz range, the FFT successfully identified the dominant frequency peaks. This allowed me to accurately calculate the heart rate, distinguishing the normal 73.8 BPM of Record 100 from the rapid 200.4 BPM indicative of the arrhythmia in Record 101.



ECG Heart Rate Calculation

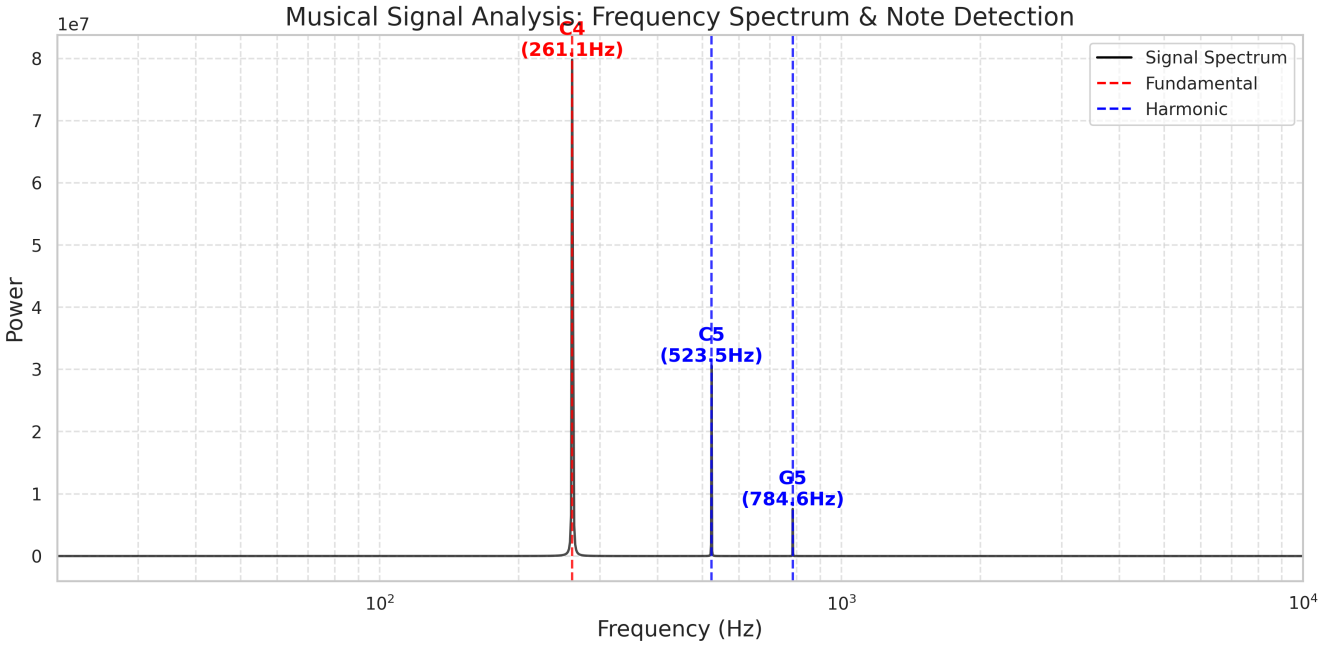
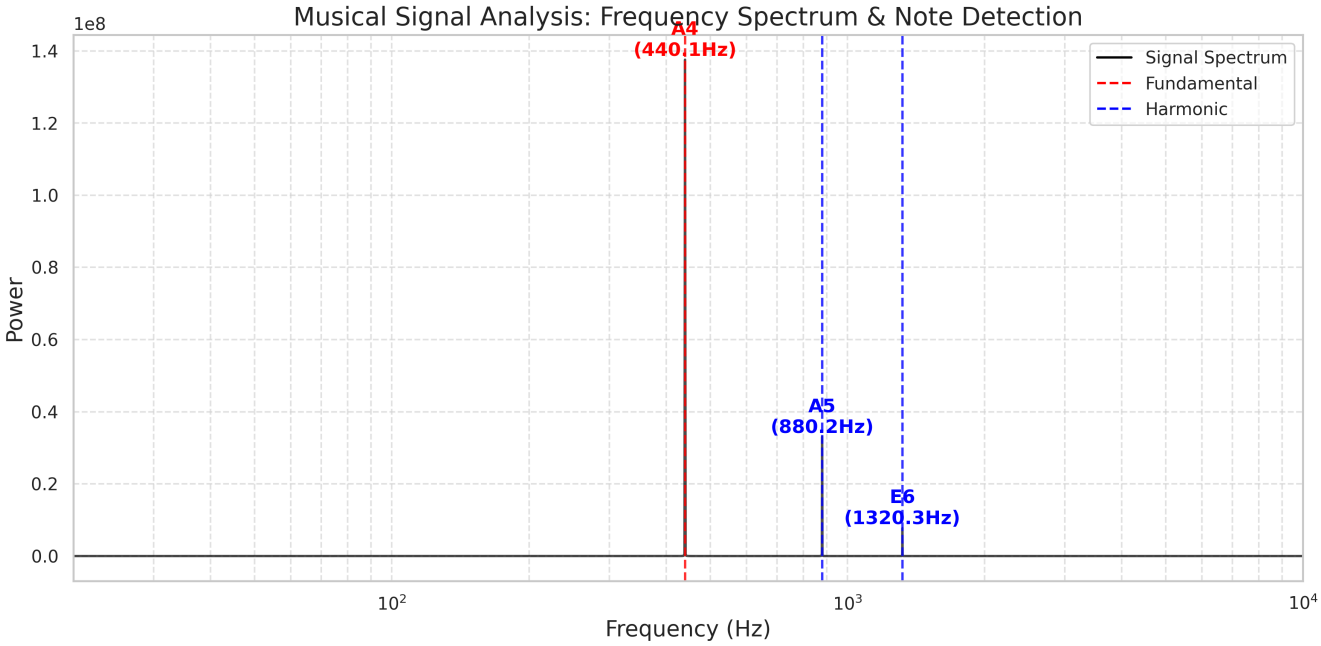
Record	Condition	Calculated BPM
100	Normal	73.83
101	Arrhythmia	200.39

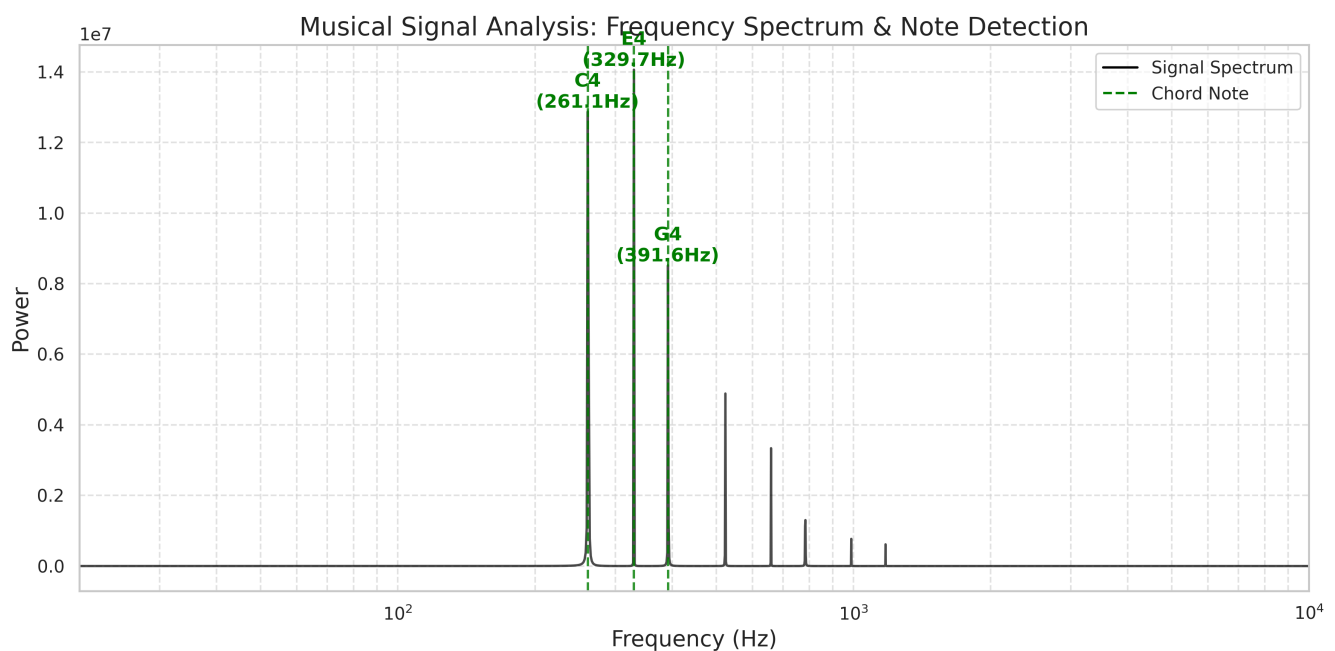
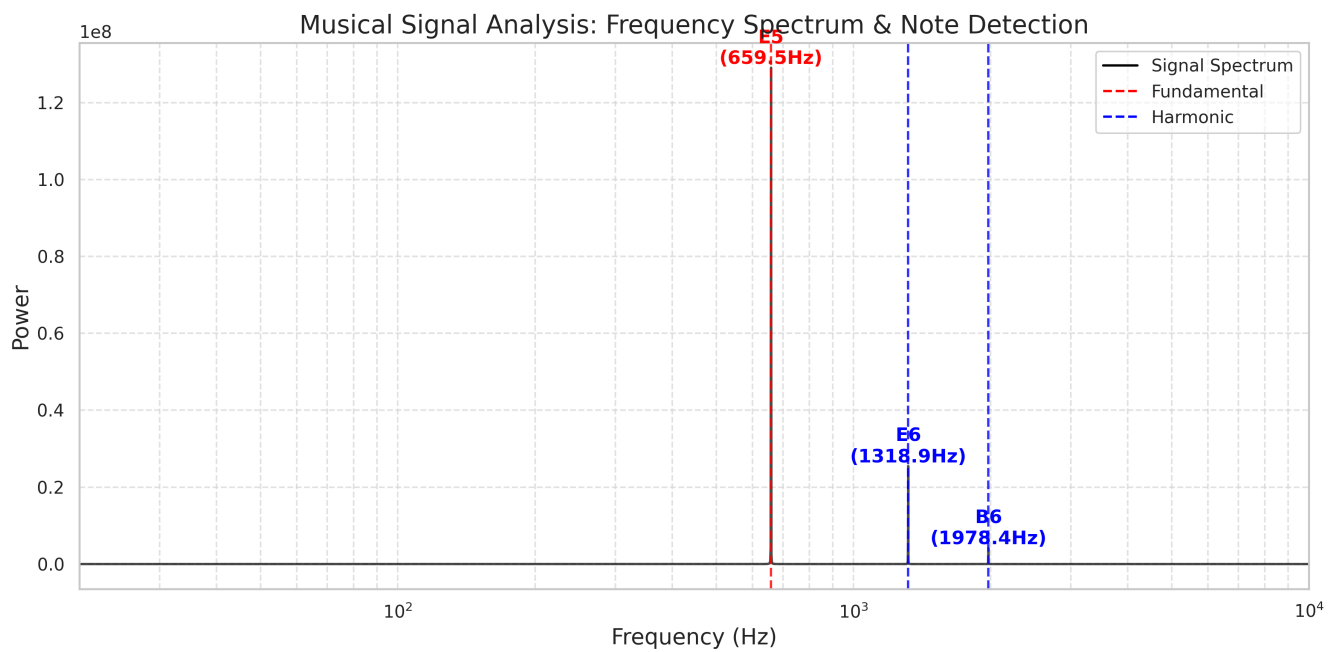
5. Synthesized Analysis: Musical Pitch Detection

Additionally, I employed synthesized audio signals containing both isolated musical notes (A4, C4, and E5) and a chord (C Major) to test pitch detection. For single notes, the algorithm successfully mapped the waveforms back to their fundamental frequencies with high precision, such as reliably identifying 440.08 Hz for the A4 note and 261.09 Hz for C4. Beyond single frequencies, the algorithm demonstrated its capability to decompose complex signals by successfully identifying the three constituent frequencies (approximately 261 Hz, 329 Hz, and 391 Hz) that make up a C Major chord.

Audio Analysis Validation

Note	Target Freq (Hz)	Detected Freq (Hz)	Identified Note	Status
A4	440.00	440.08	A4	PASS
C4	261.63	261.09	C4	PASS
E5	659.25	659.45	E5	PASS





6. Novel Concepts (Learned Outside Class)

During this research, I explored several advanced concepts beyond the scope of our standard curriculum. Foremost among these is the Cooley-Tukey Fast Fourier Transform itself. While our coursework introduced fundamental algorithmic paradigms, the FFT represents a highly specialized application of divide-and-conquer, ingeniously reducing the required complex multiplications from n^2 to $(n/2) \log_2 n$.

Furthermore, developing the iterative version of the FFT necessitated learning about in-place algorithms and bit-reversal permutations. By reversing the binary representation of the array

indices (for example, mapping 001 to 100), the data can be correctly pre-arranged. This elegant technique enables the algorithm's core mathematical operations to occur directly within the original memory footprint, significantly optimizing spatial complexity.

7. Novel Concepts (Learned In Class)

The foundational theories covered in class were instrumental in designing and evaluating this project. The entire premise relies heavily on the concept of asymptotic time complexity (Big-O notation). The stark contrast between the $O(n^2)$ and $O(n \log n)$ performance curves generated during benchmarking served as a perfect, tangible confirmation of the theoretical limits we studied.

Additionally, the project served as a deep dive into the Divide and Conquer paradigm. The recursive Cooley-Tukey FFT acts as a quintessential example of this strategy. Much like the mechanics of Merge Sort, the FFT achieves its profound algorithmic speedup by recursively fracturing a massive computational problem into smaller, manageable sub-problems.

8. Conclusion & Documentation

Consult README and docstrings for further technical details, environment setup instructions, and implementation specifics. All experimental results and visualizations have been consolidated into this report to provide a comprehensive analysis of the FFT algorithm's performance and practical utility.